Guía del Directorio de Contactos

Agregamos un botón **Eliminar** a tu directorio. Aquí tienes qué cambió y qué hace cada línea de tus tres archivos, explicado paso a paso y sin dar nada por sabido.

Qué cambió Cómo trabajan juntos los archivos app.py index.html style.css
Qué pasa al presionar Eliminar Cómo probarlo Glosario

Qué cambió

Se modificaron los tres archivos. Cada uno aporta una pieza distinta del botón:

Archivo	Qué se agregó	Líneas resaltadas
`app.py`	Una ruta nueva, `/eliminar`, que borra el contacto en la base de datos.	14
`index.html`	El botón rojo “Eliminar” junto a “Editar”, con una ventana de confirmación.	10
`style.css`	Los colores y el acomodo del botón nuevo.	28

En todos los códigos de esta guía, las líneas que cambiaron se ven así:

```
12 +    Línea nueva o modificada (fondo amarillo y signo +)
13      Línea que quedó igual
```

Cómo trabajan juntos los archivos

Piensa en un restaurante. `index.html` es el menú y la mesa: lo que ve el cliente. `style.css` es la decoración del lugar. `app.py` es la cocina: recibe los pedidos, consulta la despensa (la base de datos MySQL) y devuelve el platillo listo.

Flask es estricto con las carpetas. Tus archivos deben estar organizados así:

```
directorio/
├── app.py
├── templates/
│   └── index.html
└── static/
    └── style.css
```

Si pones `index.html` o `style.css` en la misma carpeta que `app.py`, Flask no los encontrará. Los HTML van en `templates` y los archivos fijos (CSS, imágenes) van en `static`.

`app.py` — la cocina

Es el programa en Python. Recibe las visitas a cada dirección, habla con MySQL y decide qué página mostrar. Lo vemos por bloques.

Encabezado del archivo

```
1  # app.py
2  # ------------------------------------------------------
3  # Programa en Flask que se conecta a MySQL, crea la base
4  # de datos y tabla si no existen, y permite dar de alta
5  # registros en el directorio.
6  # ------------------------------------------------------
```

Las líneas que empiezan con `#` son **comentarios**: notas para las personas. Python las ignora por completo. Aquí solo dicen para qué sirve el archivo.

Importar herramientas y crear la aplicación

```
from flask import Flask, render_template, request, redirect, url_for, abort
import mysql.connector
from mysql.connector import Error

app = Flask(__name__)
```

`import` significa “trae una herramienta que otra persona ya programó”. Aquí traemos:

- `Flask` : lo que crea tu sitio web.
- `render_template` : muestra una página HTML.
- `request` : lee lo que la persona escribió en un formulario.
- `redirect` y `url_for` : mandan al navegador a otra dirección.
- `abort` : muestra un error (por ejemplo, el 404 de “página no encontrada”).
- `mysql.connector` : permite que Python hable con la base de datos MySQL.

`app = Flask(__name__)` crea tu aplicación y la guarda en la variable `app` . Todo lo demás se “cuelga” de ella.

Datos para entrar a MySQL

```
# ------------------------------------------------------
# DATOS DE CONEXIÓN A MYSQL
# ------------------------------------------------------
DB_CONFIG = {
    "host": "localhost",
    "user": "coram",
    "password": "TU_CONTRASEÑA",
    "database": "directorio"
}
```

`DB_CONFIG` es un **diccionario**: una lista de pares *nombre → valor*. Guarda dónde está MySQL (`host`), con qué usuario y contraseña entrar, y el nombre de la base de datos.

> En esta guía la contraseña aparece como `TU_CONTRASEÑA` para que no quede escrita en un documento que podrías compartir. En tu archivo `app.py` sigue tu contraseña real.

Preparar la base de datos

```
25    def inicializar_bd():
26        """Crea la base de datos y la tabla si no existen."""
27        try:
28            conexion = mysql.connector.connect(
29                host=DB_CONFIG["host"],
30                user=DB_CONFIG["user"],
31                password=DB_CONFIG["password"]
32            )
33            cursor = conexion.cursor()
34            # Crear base de datos si no existe
35            cursor.execute("CREATE DATABASE IF NOT EXISTS directorio")
36            cursor.execute("USE directorio")
37            # Crear tabla si no existe
38            cursor.execute("""
39                CREATE TABLE IF NOT EXISTS contactos (
40                    id INT AUTO_INCREMENT PRIMARY KEY,
41                    nombre VARCHAR(100) NOT NULL,
42                    telefono VARCHAR(20),
43                    correo VARCHAR(100),
44                    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
45                )
46            """)
47            conexion.commit()
48            cursor.close()
49            conexion.close()
50            print("Base de datos y tabla listas.")
51        except Error as e:
52            print(f"Error al inicializar la BD: {e}")
```

Una **función** (`def`) es un bloque de instrucciones con nombre. Esta se ejecuta una sola vez, al arrancar el programa (lo verás al final del archivo).

Hace tres cosas: se conecta a MySQL, crea la base `directorio` si no existe y crea la tabla `contactos` si no existe. La tabla tiene columnas: `id` (un número que sube solo), `nombre` , `telefono` , `correo` y la fecha en que se guardó.

`commit()` es “guardar de verdad los cambios”. `try` / `except` significa “intenta esto; si falla, no te caigas: muestra el error”.

Abrir una conexión cuando se necesite

```
55    def obtener_conexion():
56        """Crea y devuelve una conexión a la base de datos MySQL."""
57        conexion = mysql.connector.connect(**DB_CONFIG)
58        return conexion
```

Cada vez que una ruta necesita hablar con la base de datos, llama a esta función. `**DB_CONFIG` "desempaca" el diccionario de arriba y lo usa como datos de entrada, así no repites usuario y contraseña por todos lados.

Ruta "/": la página principal

```
61    @app.route("/")
62    def index():
63        """Página principal: formulario y lista de registros."""
64        conexion = obtener_conexion()
65        cursor = conexion.cursor(dictionary=True)
66        cursor.execute("SELECT * FROM contactos ORDER BY id DESC")
67        contactos = cursor.fetchall()
68        cursor.close()
69        conexion.close()
70        return render_template("index.html", contactos=contactos, contacto_editar=N
```

Una **ruta** conecta una dirección web con una función: `@app.route("/")` dice "cuando alguien entre a la página principal, ejecuta `index()` ".

Dentro: abre la conexión, pide a MySQL todos los contactos (`SELECT * ... ORDER BY id DESC` , los más nuevos primero), los guarda en la lista `contactos` , cierra la conexión y le entrega todo a `index.html` . Con `contacto_editar=None` le avisa al HTML: "no estamos editando a nadie", así que muestra el formulario vacío.

Con `dictionary=True` cada fila llega como diccionario, por eso en el HTML puedes escribir `c.nombre` .

Ruta "/editar/número": cargar un contacto en el formulario

```
73    @app.route("/editar/<int:id>")
```

```
74    def editar(id):
75        """Selecciona un contacto y lo carga en el formulario para modificarlo."""
76        conexion = obtener_conexion()
77        cursor = conexion.cursor(dictionary=True)
78
79        # Contacto seleccionado
80        cursor.execute("SELECT * FROM contactos WHERE id = %s", (id,))
81        contacto_editar = cursor.fetchone()
82
83        # Lista completa (para seguir mostrando la tabla)
84        cursor.execute("SELECT * FROM contactos ORDER BY id DESC")
85        contactos = cursor.fetchall()
86
87        cursor.close()
88        conexion.close()
89
90        if contacto_editar is None:
91            abort(404)  # el id no existe
92
93        return render_template("index.html", contactos=contactos,
94                               contacto_editar=contacto_editar)
```

`<int:id>` es una parte variable de la dirección: si visitas `/editar/5`, la variable `id` vale 5.

Busca ese contacto (`fetchone` trae una sola fila), también trae la lista completa para seguir mostrando la tabla, y si el id no existe responde con `abort(404)`. El `%s` es un espacio reservado que MySQL rellena de forma segura con el valor de `(id,)`; nunca pegues valores directo dentro del texto SQL.

Ruta “/actualizar”: guardar los cambios

```
97    @app.route("/actualizar/<int:id>", methods=["POST"])
98    def actualizar(id):
99        """Guarda los cambios del contacto seleccionado (UPDATE)."""
100       nombre = request.form["nombre"]
101       telefono = request.form["telefono"]
102       correo = request.form["correo"]
103
104       conexion = obtener_conexion()
105       cursor = conexion.cursor()
106       sql = "UPDATE contactos SET nombre = %s, telefono = %s, correo = %s WHERE
107       cursor.execute(sql, (nombre, telefono, correo, id))
```

```
        conexion.commit()
        cursor.close()
        conexion.close()

        return redirect(url_for("index"))
```

`methods=["POST"]` indica que esta ruta solo acepta formularios enviados con el método POST (el que se usa para enviar datos). `request.form["nombre"]` lee lo que escribiste en la casilla llamada `nombre`.

`UPDATE ... SET ... WHERE id = ...` cambia los datos de **ese** contacto. Al final `redirect` te regresa a la página principal.

Ruta "/agregar": crear un contacto nuevo

```
@app.route("/agregar", methods=["POST"])
def agregar():
    """Inserta un nuevo registro en la tabla contactos."""
    nombre = request.form["nombre"]
    telefono = request.form["telefono"]
    correo = request.form["correo"]

    conexion = obtener_conexion()
    cursor = conexion.cursor()
    sql = "INSERT INTO contactos (nombre, telefono, correo) VALUES (%s, %s, %s)
    valores = (nombre, telefono, correo)
    cursor.execute(sql, valores)
    conexion.commit()
    cursor.close()
    conexion.close()

    return redirect(url_for("index"))
```

Mismo patrón: leer el formulario, conectarse, ejecutar un `INSERT INTO` (agrega una fila), `commit`, cerrar y volver a la página principal.

NUEVO — Ruta "/eliminar": borrar un contacto cambió

```
+ @app.route("/eliminar/<int:id>", methods=["POST"])
```

```
135 + def eliminar(id):
136 +     """Elimina de la tabla el contacto con el id recibido (DELETE)."""
137 +     conexion = obtener_conexion()
138 +     cursor = conexion.cursor()
139 +     sql = "DELETE FROM contactos WHERE id = %s"
140 +     cursor.execute(sql, (id,))
141 +     conexion.commit()
142 +     cursor.close()
143 +     conexion.close()
144 +
145 +     return redirect(url_for("index"))
```

Esta es la función nueva. Línea por línea:

- `@app.route("/eliminar/<int:id>", methods=["POST"])` : la dirección será, por ejemplo, `/eliminar/5` , y solo acepta POST.
- `def eliminar(id):` : la función recibe el número del contacto a borrar.
- `sql = "DELETE FROM contactos WHERE id = %s"` : la orden de MySQL para borrar. El `WHERE id = ...` es **muy importante**: sin él se borrarían todos los contactos.
- `cursor.execute(sql, (id,))` : ejecuta la orden. La coma en `(id,)` es necesaria: así Python entiende que es una lista de un solo elemento.
- `conexion.commit()` : confirma el borrado. Sin esta línea MySQL lo descarta.
- `return redirect(url_for("index"))` : regresa a la página principal, que ya mostrará la lista sin ese contacto.

¿Por qué POST y no un enlace normal? Un enlace se abre con GET, y cualquier cosa puede “visitar” un enlace sin querer (el navegador al precargar páginas, un buscador, etc.). Para acciones que borran datos es más seguro exigir un formulario POST.

Arrancar el programa

```
148    if __name__ == "__main__":
149        inicializar_bd()  # Se asegura de que la BD y tabla existan
150        app.run(debug=True)
```

`if __name__ == "__main__":` significa “solo haz esto si ejecuté este archivo directamente”. Primero prepara la base de datos y luego enciende el servidor con `app.run`

`app.run(debug=True)` reinicia el servidor solo cada vez que guardas un cambio. Es cómodo para aprender, pero no se debe usar en un sitio publicado en internet.

`index.html` — lo que ve la persona

Es una **plantilla**: HTML normal mezclado con instrucciones de Jinja (`{% ... %}` y `{{ ... }}`) que Flask rellena con datos reales antes de enviarla al navegador.

Cabecera de la página (`<head>`)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Directorio de Contactos</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}"
</head>
```

HTML describe el contenido de la página con **etiquetas** como `<head>` . La cabecera guarda información *sobre* la página: el idioma (`lang="es"`), los acentos correctos (`UTF-8`), el título de la pestaña y el enlace a la hoja de estilos.

Lo que va entre `{{ }}` no es HTML: es **Jinja**, el lenguaje de plantillas de Flask. `{{ url_for('static', filename='style.css') }}` se convierte en la ruta real de tu archivo CSS.

Inicio del cuerpo

```
<body>

    <div class="contenedor">
        <h1>📇 Directorio de Contactos</h1>
```

`<body>` es lo que se ve en pantalla. El `<div class="contenedor">` es una caja que

agrupa todo; su `class` es el "nombre" con el que el CSS la reconoce para darle color y tamaño. `<h1>` es el título principal.

El formulario (sirve para agregar y para modificar)

```
<!-- ==================== FORMULARIO ==================== -->
{% if contacto_editar %}
<h2>Modificar contacto #{{ contacto_editar.id }}</h2>
<form action="{{ url_for('actualizar', id=contacto_editar.id) }}" metho
{% else %}
<h2>Agregar nuevo contacto</h2>
<form action="{{ url_for('agregar') }}" method="POST" class="formulari
{% endif %}

    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre" required
           value="{{ contacto_editar.nombre if contacto_editar else ''

    <label for="telefono">Teléfono:</label>
    <input type="text" id="telefono" name="telefono" required
           value="{{ contacto_editar.telefono if contacto_editar else

    <label for="correo">Correo electrónico:</label>
    <input type="email" id="correo" name="correo" required
           value="{{ contacto_editar.correo if contacto_editar else ''

    <button type="submit">{{ 'Actualizar' if contacto_editar else 'Guar

    {% if contacto_editar %}
    <a href="{{ url_for('index') }}" class="cancelar">Cancelar</a>
    {% endif %}
</form>
```

Las etiquetas `{% if %}`, `{% else %}` y `{% endif %}` son decisiones de Jinja. Si Flask mandó un `contacto_editar`, la página dice "Modificar contacto" y el formulario envía los datos a `actualizar`. Si no, dice "Agregar nuevo contacto" y envía a `agregar`.

Cada `<input>` es una casilla. `name` es el nombre con el que Python la lee (`request.form["nombre"]`), `required` obliga a llenarla, y `value` la rellena con los datos actuales cuando estás editando. El botón cambia de texto ("Guardar" o "Actualizar") y aparece un enlace "Cancelar" solo al editar.

Esta parte no cambió.

Encabezado de la tabla

```
<!-- ==================== LISTA DE CONTACTOS ==================== -->
<h2>Contactos guardados</h2>

{% if contactos %}
<table class="tabla">
    <tr>
        <th>ID</th>
        <th>Nombre</th>
        <th>Teléfono</th>
        <th>Correo</th>
        <th>Acciones</th>
    </tr>
```

Si existen contactos (`{% if contactos %}`), se dibuja una tabla. `<tr>` es una fila, `<th>` una celda de encabezado. La columna “Acciones” es donde viven los botones Editar y, ahora, Eliminar.

Las filas de la tabla — aquí está el botón nuevo cambió

```
    {% for c in contactos %}
    <tr {% if contacto_editar and contacto_editar.id == c.id %}class="s
        <td>{{ c.id }}</td>
        <td>{{ c.nombre }}</td>
        <td>{{ c.telefono }}</td>
        <td>{{ c.correo }}</td>
+       <td>
+           <div class="acciones">
+               <a href="{{ url_for('editar', id=c.id) }}" class="btn-e
+
+               <form action="{{ url_for('eliminar', id=c.id) }}" metho
+                     onsubmit="return confirm('¿Seguro que quieres eli
+                   <button type="submit" class="btn-eliminar">🗑 Elim
+               </form>
+           </div>
+       </td>
    </tr>
```

`{% for c in contactos %}` repite el bloque una vez por cada contacto, y en cada vuelta `c` es un contacto distinto. `<td>` es una celda de datos.

Qué cambió: antes, la celda de acciones tenía solo el enlace Editar en una línea:

```
<td><a href="{{ url_for('editar', id=c.id) }}" class="btn-editar">🖍️ Editar</a></td>
```

Ahora la celda contiene:

- `<div class="acciones">` : una caja que permite poner los dos botones lado a lado (el estilo está en el CSS).
- El enlace **Editar**, igual que antes.
- Un `<form>` con el botón **Eliminar**. Se usa un formulario porque un enlace no puede enviar POST, y nuestra ruta de Python lo exige.
- `action="{{ url_for('eliminar', id=c.id) }}"` : genera la dirección de cada fila, por ejemplo `/eliminar/5` .
- `onsubmit="return confirm('...')"` : antes de enviar, el navegador muestra una ventana “Aceptar / Cancelar”. Si presionas Cancelar, el formulario no se envía y no se borra nada. Esto evita borrar por un clic accidental.
- `<button type="submit">` : el botón que envía el formulario.

> El mensaje de confirmación no incluye el nombre del contacto a propósito: si un nombre llevara un apóstrofo (por ejemplo O'Brien), rompería el código de la ventana de confirmación.

Cierre de la tabla y mensaje de “sin contactos”

```
70                {% endfor %}
71            </table>
72            {% else %}
73            <p>No hay contactos registrados todavía.</p>
74            {% endif %}
75
76        </div>
```

```
77
78    </body>
79    </html>
```

`{% endfor %}` termina la repetición. Si no hay contactos, el `{% else %}` muestra un mensaje amable en vez de una tabla vacía. Después se cierran las cajas y la página. Esta parte no cambió.

`style.css` — la decoración

No cambia lo que dice la página, solo cómo se ve: colores, tamaños y espacios.

Fondo de la página y caja central

```
1    /* style.css - Estilos básicos para el directorio */
2
3    body {
4        font-family: Arial, sans-serif;
5        background-color:#d7d7f4;
6        margin: 0;
7        padding: 0;
8        display: flex;
9        justify-content: center;
10   }
11
12   .contenedor {
13       background-color: #cf5e5e;
14       max-width: 600px;
15       width: 100%;
16       margin: 40px 20px;
17       padding: 30px;
18       border-radius: 10px;
19       box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
20   }
```

CSS cambia el aspecto de lo que escribiste en HTML. Cada bloque tiene un **selector** (a quién se aplica) y, entre llaves, **propiedades** (qué se le cambia).

`body` es toda la página: tipo de letra y color de fondo. `.contenedor` (el punto significa

"los elementos con `class="contenedor"` ") es la caja central: ancho máximo, color, esquinas redondeadas y una sombra suave.

Títulos

```
h1 {
    text-align: center;
    color: #487aad;
}

h2 {
    color: #34495e;
    border-bottom: 2px solid #eee;
    padding-bottom: 5px;
    margin-top: 30px;
}
```

`h1` y `h2` sin punto: así se apunta a *todas* las etiquetas de ese tipo. Definen color, la línea inferior de los subtítulos y los espacios.

Formulario y su botón

```
.formulario {
    display: flex;
    flex-direction: column;
}

.formulario label {
    margin-top: 10px;
    font-weight: bold;
    color: #555;
}

.formulario input {
    padding: 8px;
    margin-top: 5px;
    border: 1px solid #ccc;
    border-radius: 5px;
}

.formulario button {
    margin-top: 20px;
    padding: 10px;
```

```
        background-color: #3498db;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
        font-size: 16px;
    }

    .formulario button:hover {
        background-color: #2980b9;
    }
```

Acomoda las casillas una debajo de otra (`flex-direction: column`), da estilo a etiquetas y casillas, y al botón azul. `:hover` significa “cuando el mouse está encima”: el botón se oscurece un poco.

Tabla

```
    .tabla {
        width: 100%;
        border-collapse: collapse;
        margin-top: 10px;
    }

    .tabla th, .tabla td {
        border: 1px solid #ddd;
        padding: 8px;
        text-align: left;
    }

    .tabla th {
        background-color: #3498db;
        color: white;
    }

    .tabla tr:nth-child(even) {
        background-color: #f2f2f2;
    }
```

Bordes, espacios y color de encabezado. `tr:nth-child(even)` pinta de gris claro las filas pares para que sea más fácil leer.

Botón Editar (fila seleccionada)

Botón Editar y fila seleccionada

```
/* ---------- Edición de contactos ---------- */

.btn-editar {
    display: inline-block;
    padding: 4px 10px;
    background-color: #f39c12;
    color: white;
    text-decoration: none;
    border-radius: 5px;
    font-size: 14px;
}

.btn-editar:hover {
    background-color: #d68910;
}

.cancelar {
    margin-top: 10px;
    text-align: center;
    color: #333;
    text-decoration: underline;
}

/* Fila del contacto que se está modificando */
.tabla tr.seleccionado,
.tabla tr.seleccionado:nth-child(even) {
    background-color: #fff3cd;
    font-weight: bold;
}
```

El botón naranja `.btn-editar`, el enlace “Cancelar” y el color amarillo de la fila que estás modificando. Esta parte no cambió; sirve de modelo para el botón nuevo.

NUEVO — Estilos para el botón Eliminar | cambió

```
+ /* ---------- Eliminación de contactos ---------- */
+
+ /* Pone los botones Editar y Eliminar uno junto al otro */
+ .acciones {
+     display: flex;
+     gap: 6px;
+     align-items: center;
+ }
```

```
+
+ /* Quita el espacio extra que los formularios traen por defecto */
+ .form-eliminar {
+     margin: 0;
+ }
+
+ .btn-eliminar {
+     padding: 4px 10px;
+     background-color: #e74c3c;
+     color: white;
+     border: none;
+     border-radius: 5px;
+     font-size: 14px;
+     cursor: pointer;
+ }
+
+ .btn-eliminar:hover {
+     background-color: #c0392b;
+ }
```

- `.acciones`: `display: flex` pone a sus hijos (Editar y Eliminar) en una misma línea; `gap: 6px` deja un espacio entre ellos; `align-items: center` los alinea verticalmente.
- `.form-eliminar`: los formularios traen un margen por defecto que desalineaba el botón. `margin: 0` lo quita.
- `.btn-eliminar`: botón rojo con letras blancas, sin borde, esquinas redondeadas y cursor de manita. Copia la forma de `.btn-editar` para que se vean como pareja.
- `.btn-eliminar:hover`: un rojo más oscuro al pasar el mouse.

Qué pasa al presionar Eliminar

1. Haces clic en 🗑 **Eliminar** en la fila de un contacto, por ejemplo el número 5.
2. El navegador muestra la ventana "¿Seguro que quieres eliminar este contacto?". Si eliges *Cancelar*, aquí termina todo y no se borra nada.
3. Si eliges *Aceptar*, el navegador envía el formulario con POST a la dirección `/eliminar/5`.

4. Flask ve que esa dirección coincide con la ruta `@app.route("/eliminar/<int:id>")` y ejecuta la función `eliminar(5)`.
5. La función le pide a MySQL `DELETE FROM contactos WHERE id = 5` y confirma con `commit()`.
6. `redirect` te devuelve a la página principal. Como la lista se vuelve a leer de la base de datos, el contacto ya no aparece.

Cómo probarlo

1. Verifica que MySQL esté encendido y que la carpeta tenga la estructura de arriba.
2. Instala las librerías, solo la primera vez:

```
pip install flask mysql-connector-python
```

3. Desde la carpeta `directorio`, ejecuta:

```
python app.py
```

4. Abre `http://127.0.0.1:5000` en tu navegador, agrega un contacto de prueba y presiona Eliminar.

> Tip para el futuro: la contraseña de la base de datos está escrita directamente en `app.py`. Si algún día subes el proyecto a GitHub o lo compartes, cámbiala antes o guárdala fuera del código.

Glosario rápido

Ruta

Una dirección de tu sitio (como `/agregar`) conectada con una función de Python.

Plantilla

Un HTML con huecos que Flask llena con datos antes de enviarlo.

GET y POST

Dos formas de pedir cosas al servidor. GET sirve para ver páginas; POST para enviar datos que cambian algo, como guardar o borrar.

SQL

El idioma para dar órdenes a la base de datos: `SELECT` lee, `INSERT` agrega, `UPDATE` modifica, `DELETE` borra.

Cursor

El "mensajero" que lleva tus órdenes SQL a MySQL y trae las respuestas.

Commit

Confirmar y guardar de forma definitiva los cambios hechos a la base de datos.

Clase (`class`)

Un nombre que se pone a una etiqueta HTML para que el CSS sepa a quién darle estilo.

Selector

La parte del CSS que dice a qué elementos se aplican los estilos, como `.btn-eliminar`.